

Math-Routed Sparse Transformer — About

Variables

P = program

$G = (V, E)$ = symbolic topology graph

K = selected neighbor budget

A_K = top-K sparse attention

$B = \lceil N/b \rceil$ = number of blocks, block size b

What The Program Is Really Called

Repository name:

Math-Routed Sparse Transformer

More exact technical name:

Topology-Routed Top-K Sparse Transformer

or:

Symbolic-Topology Routed Sparse Attention Transformer

Best clean name:

Math-Routed Sparse Transformer

because the routing is produced from mathematical expression structure.

What Concept Is Being Applied

The main concept is:

graph/topology-conditioned sparse attention

Meaning:

use a graph to decide which tokens are allowed to attend to which other tokens

In this case, the graph is not arbitrary. It is built from math structure:

expression tree
operator similarity
shape compatibility
composition
embedding similarity
local window
middle-context bridge
identity
learned pairwise scores
learned block-pair scores

So the real concept is:

neurosymbolic graph-routed attention

Exact Pipeline

The full pipeline has three modes: symbolic, learned, and block-pair.

Symbolic pipeline

math expression $\rightarrow$ symbolic graph $\rightarrow$ priority neighbor table $\rightarrow$ top-K sparse attention $\rightarrow$ transformer block

In code terms:

```
MathNode list
-> TopologyBuilder / MiddlePreservingTopologyBuilder
-> TopologyCache (offline compile)
-> neighbors (T, K), valid_i8 (T, K)
-> Triton block-token sparse attention
-> transformer block output
```

Learned pipeline

```
MathNode list
-> edge feature tensor (T, T, F) [offline]
-> LearnedTopologyScorer -> scores (T, T)
```

- > topK neighbor selection
- > PreparedTopology
- > Triton block-token sparse attention
- > transformer block output

Block-pair pipeline (v14, avoids $O(N^2)$)

```
MathNode list
-> block features (B, F)
-> HeuristicBlockTopologyBuilder: score (B, B) block pairs
-> expand selected blocks -> token neighbors (T, K)
-> Triton block-token sparse attention
-> transformer block output
```

What It Is Not

It is **not** just a normal transformer.

A normal transformer does:

$$A = \text{softmax}(QK^\top)$$

Every token can attend to every token:

$$O(T^2)$$

This system does:

$$A_i = \text{softmax}(Q_i K_{N_i}^\top)$$

where:

$$N_i = \text{TopK}(\text{symbolic/topological neighbors of } i)$$

So each token only attends to selected neighbors:

$$O(TK)$$

And with block topology (v14):

$$N_i = \text{expand}(\text{TopK}_B(\text{block neighbors of block}(i)))$$

The topology compile cost is:

$$O(B^2) = O\left(\frac{T^2}{b^2}\right)$$

which reduces the scoring cost by a factor of b^2 .

Concept Stack

Layer	Concept
Math expressions	symbolic structure
Expression tree	AST / term graph
Relations	typed symbolic topology
Middle-bridge relation	anti-lost-in-the-middle routing
Learned edge scores	trained pairwise topology policy
Block-pair scores	$O(B^2)$ scalable topology scoring
Neighbor selection	graph routing + top-K truncation
Top-K truncation	bounded compute
Attention over neighbors	sparse attention
Block-token Triton kernel	one program per (batch, head, token-block)
Prepared static topology	precompiled buffers, no lookup at inference
Dense-vs-sparse comparison	block-level systems optimization

Closest Known Concepts

Existing concept	How close
Sparse Transformer	yes, because attention is sparse
Graph Transformer	yes, because attention is graph-conditioned
Graph Neural Network	related, but not exactly
Neurosymbolic Transformer	yes, because symbolic math structure routes neural attention
Routing Transformer	related, because tokens are routed to subsets
Longformer/BigBird-style sparse attention	related, but the mask is math/topology-driven
Compiler-optimized attention	yes, because topology is compiled/cached before execution
Block-sparse attention	v14 pivot targets this directly

The Kernel Idea

$$G = (V, E)$$

Build symbolic graph (offline).

$$N_i = \text{TopK}_K(E_i)$$

Select bounded neighbors per token.

$$y_i = \sum_{j \in N_i} \text{softmax}(q_i k_j) v_j$$

Compute attention only over those neighbors.

One-line explanation:

Use symbolic math structure to route attention before the GPU computes it.

For the learned variant:

Train a scorer to select neighbors that maximize route quality,
then cache the result as a static neighbor table.

For the block variant:

Score B×B block pairs (B = N / block_size) to avoid O(N²) token-pair scoring,
then expand selected blocks into the token-neighbor table.

The Big Idea

This is:

compile symbolic structure into neural compute routing

That is the real concept.

Not “make a transformer faster” in general.

More precisely:

symbolic topology → bounded neural attention

The v14 extension adds:

block-pair topology → scalable bounded attention for large N

Current Engineering State

The project has crossed the prepared-static parity threshold:

$$p_{\text{blk}} = 0.941 \text{ ms} < d_{\text{blk}} = 2.398 \text{ ms}$$

at $n = 1024$, $K = 16$, trees, middle_preserving_topk.

The current best-known hot-path shape:

fixed-K symbolic topology compiled to K=16 per token
prepared static topology buffers (neighbors, valid_i8 as registered buffers)
Triton block-token sparse attention (block_t=2)
standard LayerNorm + QKV + out_proj + FFN around it

The remaining overhead distribution:

$$\frac{p_{\text{attn}}}{p_{\text{blk}}} \approx 34\% \quad (\text{attention kernel})$$

$$\frac{p_{\text{non}}}{p_{\text{blk}}} \approx 66\% \quad (\text{LayerNorm, QKV, out_proj, FFN, residual})$$

The bottleneck has shifted from topology routing to standard transformer block overhead.

Learned topology quality result:

learned $K = 6$: route_acc = 0.9871

hand $K = 16$: route_acc = 0.9821

Block topology status (v14 target):

HeuristicBlockTopologyBuilder: O(B^2) scoring, no N^2 allocation
N=1024, N=2048, N=4096 topology construction: no OOM
block token expansion: compatible with existing PreparedTopology interface

Best Name Going Forward

Use:

Math-Routed Sparse Transformer

For the method/concept, say:

neurosymbolic topology-routed sparse attention

For the execution system, say:

cached top-K graph-routed Triton block-token attention

For the current development stage, say:

fixed-K symbolic + learned-block topology with prepared static sparse inference

Final canonical form:

Math-Routed Sparse Transformer

neurosymbolic topology-routed top-K sparse attention
with learned block-pair topology for scalable N